

2.5 Programming languages and Integrated Development Environments

Revise360

 Teacher guide
 Unit 2.5
 revise360.co.uk

Where this unit sits, what it covers, and how far

Where this unit sits

Qualification	GCSE Computer Science (9-1), specification code J277, from 2020
Component	Component 02: Computational thinking, algorithms and programming — 80 marks, 1 hour 30 minutes
What OCR says the component covers	computational thinking: algorithms, programming techniques, producing robust programs, computational logic and translators
This unit	topic 2.5 Programming languages and Integrated Development Environments
Checked	3 October 2026, at https://www.ocr.org.uk/qualifications/gcse/computer-science-j277-from-2020/specification-at-a-glance/

How far this has been checked. The topic numbers and titles below are the ones this course uses on its worksheets and slides. They were not re-checked against the published specification document when this map was built: the document could not be retrieved from this build environment. Check them against <https://www.ocr.org.uk/Images/558027-specification-gcse-computer-science-j277.pdf> before relying on this map for a scheme of work, and treat any finer subdivision as absent rather than assumed - no sub-point numbers are claimed anywhere in these documents, because none were verified.

Before it: 1.1 Systems architecture, 1.2 Memory and storage, 1.3 Computer networks, connections and protocols, 1.4 Network security, 1.5 Systems software, 1.6 Ethical, legal, cultural and environmental impacts of digital technology, 2.1 Algorithms, 2.2 Programming fundamentals, 2.3 Producing robust programs, 2.4 Boolean logic

After it: PY Python programming

The lesson sequence and the questions it answers

#	LESSON	THE BIG QUESTION	ACTIVITIES	MARKS
1	Language level lab p1-l01	Why might two programmers choose different languages to solve the same problem?	14	27
2	Language choice workshop p1-l02	Why might a school app and a small hardware routine need different language choices?	14	26
3	Translation studio p1-l03	Why can a CPU not simply follow a Python source file as written?	14	24
4	IDE control room p1-l04	What help would you want from the software you use to write programs?	14	23
5	Language review HQ p1-l05	Which tool or language choice would most help you build a reliable revision app?	14	28

Prior knowledge, vocabulary and skills

Assumed before the unit: the outcomes of 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 2.1, 2.2, 2.3, 2.4

Vocabulary introduced: High-level language, Low-level language, Portability, Maintainability, Compiler, Interpreter, IDE, Error diagnostics, Source code, Run-time environment

What a pupil does, not only reads: match, mcq, multi, sort

Coverage matrix

Every outcome this unit states, and whether it is taught, practised and assessed **within this unit**. Full means all three; partial means some; not covered means none of the three could be traced.

How this was worked out. The trace is derived by matching the content words of an outcome against the station text, the activities and the exam questions of the unit, not authored by hand. The words each match was made on are in `build/coverage.json`. It shows where to look; it does not certify that the alignment is right, and a full row has not been checked by a person either.

OUTCOME	LESSON	TAUGHT	PRACTISED	ASSESSED	COVERAGE
Compare high-level and low-level languages. (p1-l01-o1)	pl-l01	yes	yes	yes	Full
Explain why high-level source needs translation. (p1-l01-o2)	pl-l01	yes	yes	yes	Full
Justify language choices using project requirements. (p1-l01-o3)	pl-l01	—	—	—	Not covered
Compare development effort at different language levels. (p1-l02-o1)	pl-l02	yes	yes	yes	Full
Explain portability and hardware control. (p1-l02-o2)	pl-l02	yes	yes	yes	Full
Justify language choices using project requirements. (p1-l02-o3)	pl-l02	—	—	—	Not covered
Explain why programs need translation. (p1-l03-o1)	pl-l03	yes	yes	yes	Full
Compare how compilers and interpreters work. (p1-l03-o2)	pl-l03	yes	yes	yes	Full
Justify a translator choice using the task requirements. (p1-l03-o3)	pl-l03	yes	yes	yes	Full
Identify the four main IDE facilities. (p1-l04-o1)	pl-l04	yes	yes	yes	Full
Explain how each facility supports programming. (p1-l04-o2)	pl-l04	—	—	yes	Partial
Use editing, diagnostics and execution tools in an IDE. (p1-l04-o3)	pl-l04	yes	yes	yes	Full
Recall language levels and translator behaviour. (p1-l05-o1)	pl-l05	yes	yes	yes	Full
Select IDE tools for specific problems. (p1-l05-o2)	pl-l05	yes	yes	yes	Full
Write explanations linked to the needs of a scenario. (p1-l05-o3)	pl-l05	yes	yes	—	Partial

11 full · 2 partial · 2 not covered, of 15 outcomes.

Gaps, assumptions and overlap

- **pl-l01-o3** Justify language choices using project requirements. — cannot be traced: this lesson's station text is only in the rendered image, so there is nothing to match; nothing in this unit assesses it
- **pl-l02-o3** Justify language choices using project requirements. — cannot be traced: this lesson's station text is only in the rendered image, so there is nothing to match; nothing in this unit assesses it
- **pl-l04-o2** Explain how each facility supports programming. — cannot be traced: this lesson's station text is only in the rendered image, so there is nothing to match
- **pl-l05-o3** Write explanations linked to the needs of a scenario. — nothing in this unit assesses it

What could not be traced

The wall text of pl-l01, pl-l02, pl-l03, pl-l04, pl-l05 is painted into the 360 image and held nowhere as text, so an outcome in those lessons can only be traced through its activities and its exam questions. Read the rows above for them with that in mind.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.